

Reviewer Pack — Minimal Hodge Index of Affine Schemes as Monotone Circuit Lo...

Entry 604199f16853 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 05:04:42 UTC

Minimal Hodge Index of Affine Schemes as Monotone Circuit Lower Bound for k-CLIQUE

Entry ID: 604199f16853 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 05:04:42 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Affine Schemes **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

{'part1': 'The minimal Hodge index of an affine scheme associated with a k-Clique instance is upper bounded by a function of the size of the clique, i.e., $E[\text{HodgeIndex}(\text{instance})] \leq \Theta(n^{\{k/2\}})$.' , 'part2': 'For all instances with k-CLIQUE property P, the resolution proof length Q satisfies $Q \geq 2^n / (\text{HodgeIndex}(\text{instance}))^2$.' , 'part3': 'Here, HodgeIndex is computed using a constructive mapping from the clique instance to an affine scheme.' }

Rationale (proposer's reasoning):

{'part1': 'The minimal Hodge index could potentially capture the geometric complexity of k-CLIQUE instances, providing a novel invariant for monotone circuit lower bounds.' , 'part2': 'Affine schemes are known to have rich geometric structures that may correspond to the complexity of computational problems.' , 'part3': 'This conjecture links algebraic geometry with complexity theory, potentially leading to new insights into the hardness of k-CLIQUE.' }

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 14aaf281c9d58744

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if the average minimal Hodge index of k-Clique instances (over 30 seeds) is less than or equal to $n^{(k/2)}$, and all computed resolution proof lengths are greater than or equal to $2^n / (\text{HodgeIndex}(\text{instance}))^2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Hodge index AND Affine Schemes AND Monotone Circuit Complexity
- k-CLIQUE AND Hodge index AND Affine Scheme resolution - constructive mapping FROM clique instance TO Affine Scheme AND complexity lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(b)
    M = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M[i], M[max_row] = M[max_row], M[i]
        factor = M[i][i]
```

```

    for j in range(n):
        M[i][j] /= factor
    b[i] /= factor
    for j in range(i+1, n):
        factor = M[j][i]
        for k in range(n):
            M[j][k] -= factor * M[i][k]
        b[j] -= factor * b[i]
x = [0 for _ in range(n)]
for i in range(n-1, -1, -1):
    x[i] = b[i]
    for j in range(i+1, n):
        x[i] -= M[i][j] * x[j]
return x

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    k = random.randint(2, min(n-1, 6))
    clique = set(random.sample(range(n), k))

    # Constructive mapping from clique to affine scheme
    A = [[0 for _ in range(k)] for _ in range(k)]
    b = [0 for _ in range(k)]
    for i in range(k):
        for j in range(i+1, k):
            if (i, j) in clique or (j, i) in clique:
                A[i][j] = 1
                A[j][i] = 1
                b[i] += 1
                b[j] += 1

    # Solve linear system to get Hodge index
    try:
        x = gaussian_elimination(A, b)
        hodge_index = sum(x)
    except Exception as e:
        return {
            "metric_name": "Hodge Index and Resolution Length",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Mapping error: {e}"
        }

    # Calculate resolution proof length
    try:
        resolution_length = 2**n / (hodge_index ** 2)
    except ZeroDivisionError:
        return {
            "metric_name": "Hodge Index and Resolution Length",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Resolution length calculation error: Division by zero"
        }

```

```

# Check conjectures
conjecture_holds = hodge_index <= n ** (k / 2) and resolution_length >= 2**n / (hodge_index **
counterexample = "" if conjecture_holds else f"Hodge Index: {hodge_index}, Resolution Length:

return {
    "metric_name": "Hodge Index and Resolution Length",
    "metric_value": hodge_index,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30*31, 31))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if "conjecture_holds" in r and r["
    support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"])

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0 support_fraction=1")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if "conjecture_holds" not in r or not r["
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

terexample': 'Mapping error: division by zero'}
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
TRIAL: {'metric_name': 'Hodge Index and Resolution Length', 'metric_value': inf, 'instances_tested':
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c6a54f5.py", line 130, in <module>
    first_failing_seed = next(r["seed"] for r in results if "conjecture_holds" not in r or not r["conject
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c6a54f5.py", line 130, in <genexpr>

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for further testing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14291
2	propose	ollama_remote	glm4:latest	0	0	13622
3	preregistration	ollama_remote	glm4:latest	0	0	9725
4	novelty	ollama_remote	glm4:latest	0	0	8323
5	novelty	ollama_remote	glm4:latest	0	0	8825
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22570
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11865
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11368
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13899
10	judge	ollama_remote	glm4:latest	0	0	11577

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126066 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/604199f16853.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/604199f16853.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/604199f16853.tar.gz` (if generated)